

Relatório Analítico: Sistema de Matchmaking para Jogos Online

Miguel Fernandes de Alemida Gomes

Jordana Martinelli

13 de maio de 2026

1 Implementação dos Algoritmos de Ordenação

Insertion Sort: O algoritmo foi implementado iterando sobre o array e movendo os elementos maiores para a direita. Essa abordagem *in-place* exige, no pior caso, cerca de $N(N - 1)/2$ comparações e um alto volume de trocas na memória, o que justifica sua lentidão em arrays grandes e desordenados.

Merge Sort: Implementado utilizando o paradigma de Divisão e Conquista através da recursão. O array é dividido sucessivamente pela metade até atingir o caso base (sub-arrays de tamanho 1). Em seguida, no retorno da recursão, a função **merge** intercala essas divisões, ordenando os elementos corretamente com o auxílio de memória dinâmica, garantindo estabilidade e alta eficiência.

2 Estratégia para Formação de Grupos

A ordenação prévia simplificou drasticamente o problema de *matchmaking*. Como a lista já se encontra ordenada pelo *score*, jogadores com níveis semelhantes ficam alocados em posições adjacentes.

Para formar um grupo de tamanho m , o sistema avalia a diferença de *score* entre o primeiro e o m -ésimo jogador da janela atual. Se a diferença for $\leq \Delta$ (**delta**), o grupo é considerado válido. Caso contrário, a janela desliza uma posição (analisando do segundo ao $m + 1$ -ésimo jogador), repetindo a verificação até encontrar um grupo válido ou esgotar os elementos do array.

3 Análise de Custo Computacional

Abaixo está o detalhamento da complexidade de tempo (*Big-O*) para as principais operações do sistema:

- **Inserção** – $\mathcal{O}(1)$: Inserimos sempre no final do array estático (controlado pelo contador de tamanho), sem necessidade de deslocamento ou reordenação imediata.

- **Remoção** – $\mathcal{O}(N)$: É necessário realizar uma busca linear para encontrar o ID e, em seguida, deslocar todos os elementos à sua direita para preencher o espaço removido.
- **Ordenação (Insertion Sort)** – $\mathcal{O}(N^2)$: Devido aos laços aninhados para percorrer a lista e deslocar os elementos, o tempo de execução cresce quadraticamente com o tamanho do array.
- **Ordenação (Merge Sort)** – $\mathcal{O}(N \log N)$: A divisão da lista cria $\log N$ níveis de recursão, e a etapa de fusão (*merge*) percorre os N elementos de forma linear em cada nível.
- **Formação de Grupo** – $\mathcal{O}(N)$: Graças à ordenação prévia, a janela deslizante precisa percorrer o array no máximo uma única vez.
- **Recuperação dos Jogadores** – $\mathcal{O}(N)$: O sistema itera sobre a lista atual copiando todos os elementos para um novo array alocado dinamicamente.

4 Comparação Experimental de Desempenho

Para validar a teoria da complexidade computacional, foi desenvolvido um teste de *benchmark* populando o sistema com dados gerados aleatoriamente. O tempo foi cronometrado utilizando a biblioteca `<chrono>` de C++. Os testes submeteram os mesmos arrays desordenados a ambos os algoritmos, variando a quantidade de jogadores (N).

Tabela 1: Comparativo de tempo de execução entre algoritmos de ordenação.

Tamanho do Array (N)	Insertion Sort (ms)	Merge Sort (ms)
1.000	13.60	2.14
5.000	320.86	12.89
10.000	1358.56	26.58
20.000	5303.12	55.63

Discussão dos Resultados

Os resultados empíricos refletem exatamente a teoria da complexidade esperada. Em amostras pequenas ($N = 1000$), a diferença é notável, porém ambos performam em frações de segundo. Contudo, ao escalar a entrada, o comportamento dos algoritmos diverge drasticamente.

Ao dobrar a quantidade de jogadores de 10.000 para 20.000, o tempo do **Merge Sort** foi de aproximadamente 26.5ms para 55.6ms. Esse crescimento é proporcional a cerca de duas vezes o valor anterior, o que caracteriza perfeitamente a curva linear-logarítmica $\mathcal{O}(N \log N)$.

Por outro lado, no mesmo cenário, o **Insertion Sort** saltou de 1358ms para 5303ms. O tempo de execução quadruplicou (aumento de 4x) ao dobrarmos a entrada ($2^2 = 4$), comprovando sua natureza quadrática $\mathcal{O}(N^2)$. Conclui-se que o Merge Sort é arquiteturalmente melhor que o do Insertion Sort uma vez que na medida que há o aumento do tamanho do Array, o Merge Sort possui menor Complexidade.